HLD Level 1.7

Messaging: Queues, Pub/Sub & Event Streaming - C++ / Systems Interview Notes

> **Goal: choose asynchronous messaging appropriately and reason about ACKs, retries, duplicates, ordering, partitions, consumer groups, DLQs and backpressure.**

1. Why messaging?

Messaging decouples producers from consumers, absorbs temporary bursts, supports independent scaling and isolates temporary failures.

```
A -> Queue/Broker -> B
A can continue without waiting for B
```

2. Producer-consumer

A producer creates messages/jobs; a broker stores/routes them; consumers process them. This is the distributed version of your C++ producer-consumer pattern.

```
Producer -> [Queue] -> Consumer
```

3. When not to use it

A direct synchronous call is often simpler when the caller needs the result immediately. Brokers add operational complexity, latency, retries, duplicate handling and ordering questions.

4. Work queues

Jobs are distributed among a pool of workers; one logical job is normally handled by one worker in that group. Examples: image processing, email, report generation.

```
Producer -> Queue -> Worker A / Worker B / Worker C
```

5. Pub/Sub

One published event fans out to multiple independent subscribers. Example: OrderCreated can independently trigger inventory, analytics and notifications.

```
Publisher -> Topic -> Inventory
-> Analytics
-> Notifications
```

6. Event streaming

A retained event stream/log stores events for a configured period. Consumers track positions/offsets and may replay older events. Retention and ordered partitions are central ideas.

```
100 DeviceOnline
101 TempUpdated
102 DeviceOffline
```

7. Acknowledgements

An ACK tells the broker processing reached the required success point. If a consumer crashes before ACK, the message may be redelivered.

```
Broker -> Consumer -> process -> ACK
```

8. At-most-once

A message is processed zero or one time under the delivery model. Avoiding retries reduces duplicates but failures can cause loss.

9. At-least-once

Retries/redelivery reduce loss but duplicates are possible. Consumers should normally tolerate duplicate delivery.

```
process Message42 -> ACK lost -> Message42 delivered again
```

10. Exactly-once caution

End-to-end exactly-once effects across brokers, databases and external systems are difficult and depend on specific transactional/idempotency guarantees. Reason explicitly about duplicates instead of casually claiming exactly once.

11. Idempotency

Repeating the same logical operation should not create an unintended duplicate side effect. Techniques include idempotency keys, processed-message IDs, unique constraints and version checks.

```
request_id=ABC123 -> already processed? -> return prior result
```

12. Retries

Use bounded retries, exponential backoff and jitter where appropriate. Immediate unlimited retries can worsen an outage.

```
attempt -> fail -> wait -> retry -> longer wait -> retry
```

13. Dead-letter queue

Messages that repeatedly fail can be isolated in a DLQ for inspection, correction or later replay instead of retrying forever.

```
Main Queue -> Consumer -> repeated failure -> DLQ
```

14. Poison messages

A malformed/problematic message may fail deterministically on every attempt. Retry limits and DLQ isolation prevent an endless failure loop.

15. Ordering

Global ordering is expensive and often unnecessary. Define the smallest ordering scope the business needs, such as per device or per account.

```
Device42: Config v10 -> v11 -> v12
```

16. Partitions

Streams are partitioned to increase parallelism. Events sharing an ordering key can be routed to the same partition to preserve relative order.

```
Topic -> P0 / P1 / P2
```

17. Partition key

A key such as device_id can preserve per-device ordering. But a hot key can create a hot partition, so distribution matters just as it does in database sharding.

```
partition = hash(device_id) % N
```

18. Consumer groups

Multiple consumers in one group cooperate on the same workload; partitions are assigned among members. A separate group can independently consume the same stream for another purpose.

```
P0,P1 -> Worker1
P2,P3 -> Worker2
```

19. Consumer lag

Lag measures how far processing is behind the producer/stream head. Growing lag means consumers cannot keep up.

```
latest offset=1,000,000
consumer=900,000
lag=100,000
```

20. Backpressure

If production permanently exceeds consumption, backlog grows. A broker absorbs bursts but cannot solve a sustained throughput mismatch.

```
producer=50k/s, consumer=40k/s -> +10k backlog/s
```

21. Bounded queues

A bounded queue forces an explicit overload policy: block/throttle producer, reject, drop newest, drop oldest, spill durably, or scale processing. The right policy comes from requirements.

22. Freshness workloads

For live camera inference, if capture is 30 FPS and inference is 20 FPS, preserving every frame makes results increasingly stale. A bounded queue with old-frame eviction can prioritize freshness. Archival recording has different requirements.

```
Capture 30 FPS -> bounded queue -> Inference 20 FPS
```

23. Batching

Batching amortizes network, serialization, transaction and disk overhead. It improves throughput but can increase latency and complicate partial failures.

```
1000 individual calls vs 10 batches x 100 messages
```

24. Durability

Ask when the producer is acknowledged: after broker memory, durable local storage, or replication? Stronger durability usually adds latency.

```
Producer -> broker memory -> durable log -> replica(s)
```

25. Retention and replay

Queues may remove acknowledged work quickly; streams may retain events by time/size policy. Replay enables rebuilding derived state, fixing consumer bugs and running new consumers.

26. Schema evolution

Messages are contracts between independently deployed producers and consumers. Field changes must preserve required backward/forward compatibility.

27. Large messages

For large video/image objects, store bytes in object storage and send a reference plus metadata through messaging rather than placing huge payloads in the broker.

```
Object Storage <- video bytes
Broker <- {video_id, object_key, metadata}
```

28. Broker failure

The messaging layer itself requires availability/durability planning. Do not treat the broker as an infallible box.

29. C++ implementation bridge

HLD can zoom into an I/O consumer thread, deserialization, bounded in-process queue, worker pool, buffer ownership, ACK timing and graceful shutdown.

```
Broker socket -> I/O thread -> bounded queue -> workers -> ACK
```

30. Graceful shutdown

Stop fetching new work, decide how queued/in-flight work is handled, complete or abandon safely according to semantics, ACK only completed work, then release connections/resources.

31. Queue vs Pub/Sub vs Stream

Work queue: distribute jobs to workers.
Pub/Sub: fan an event to independent consumers.
Event stream: retained ordered log with consumer positions/replay. Real products may overlap; describe required semantics.

32. Telemetry example

Ingestion publishes telemetry to a stream; storage, alerts and analytics consume independently and scale separately.

```
Devices -> Ingestion -> Stream -> Storage
-> Alerts
-> Analytics
```

33. Image-job example

Upload bytes to object storage and enqueue a small processing job/reference. Workers consume jobs independently.

```
Upload -> Object Storage
+-> Work Queue -> Worker1/Worker2/Worker3
```

34. Interview checklist

1. Why async instead of direct call?
2. Work queue, pub/sub or retained stream?
3. Can messages be lost?
4. Can messages be duplicated?
5. When is ACK sent?
6. Is processing idempotent?
7. What ordering is required?
8. Partition key?
9. Hot partition?
10. What if consumers fall behind?
11. Retry/backoff policy?
12. DLQ?
13. Retention/replay?
14. Message size/schema?
15. Graceful shutdown?

35. Common mistakes

Queue solves overload: only temporary bursts; sustained mismatch grows backlog.
At-least-once = exactly once: false.
ACK immediately on receive: can lose work if processing later fails.
Retry forever: poison messages can loop.
Global ordering everywhere: expensive/unnecessary.
More consumers always help: partitions/downstream bottlenecks can cap throughput.
Huge video in broker: usually send object reference.
Unbounded queue: hides overload until latency/storage explodes.

36. Practice scenarios

1. OrderCreated must trigger inventory, analytics and email independently. Queue or pub/sub?

2. Consumer charges a card but crashes before ACK; message is redelivered. What danger exists and how do you handle it?

3. Producer sends 50k msg/s and consumers process 40k/s. What happens after 10 minutes?

4. Per-device ordering matters but cross-device ordering does not. How can partitioning help?

5. One malformed message fails every retry. What mechanism prevents endless retry?

6. Why send a video object key instead of a 100 MB video through the broker?

7. For live camera inference, should every queued frame always be preserved? Explain.

37. 60-second revision

Queue: distribute work.
Pub/Sub: fan out events.
Stream: retained log with replay.
ACK: confirms required processing point.
At-most-once: loss possible, duplicates minimized.
At-least-once: duplicates possible.

Idempotency: repeat does not duplicate effect.
DLQ: isolate repeatedly failing messages.
Partition: ordering/parallelism unit.
Consumer group: cooperating consumers.
Lag: consumer distance behind head.
Backpressure: control producer-consumer mismatch.

38. Ready for Level 1.8 when...

You can choose queue vs pub/sub vs stream, explain ACK/redelivery and delivery guarantees, design idempotent consumers, reason about ordering/partitions/consumer groups, and handle backlog, DLQs, batching and bounded queues.